

Universidad Panamericana
Facultad de Ingeniería y Ciencias Aplicadas

Sistema de Mensajería Cifrada Distribuida

Especificación de Requisitos de Software

Estándar	Revisión	Fecha	Estado
IEEE Std 830-1998	1.0	Junio 2025	Aprobado

Fecha	Revisión	Autor	Verificado — Depto. Calidad
Junio 2025	Rev. 1.0	Eliseo Velásquez	Eliseo Velásquez Rodas

Documento validado en fecha: Junio 2025

Por el cliente / Institución	Por el equipo suministrador
Eliseo Velásquez Rodas	Eliseo Velásquez Rodas

Contenido

Contenido.....	2
1 Introducción.....	4
1.1 Propósito	4
1.2 Alcance.....	4
1.3 Personal Involucrado.....	4
1.4 Definiciones, Acrónimos y Abreviaturas	5
1.5 Referencias	6
1.6 Resumen	6
2 Descripción General.....	6
2.1 Perspectiva del Producto	6
2.2 Funcionalidad del Producto.....	7
2.3 Características de los Usuarios.....	7
2.4 Restricciones	7
2.5 Suposiciones y Dependencias	8
2.6 Evolución Previsible del Sistema	8
3 Requisitos Específicos	8
3.1 Requisitos Comunes de los Interfaces.....	8
3.1.1 Interfaces de Usuario.....	8
3.1.2 Interfaces de Hardware	9
3.1.3 Interfaces de Software	9
3.1.4 Interfaces de Comunicación	10
3.2 Requisitos Funcionales	10
3.2.1 RF-01 — Registro de Usuario con Generación de Claves RSA.....	10
3.2.2 RF-02 — Autenticación con Emisión de Token de Sesión	11
3.2.3 RF-03 — Envío de Mensaje con Cifrado E2E	11
3.2.4 RF-04 — Recepción y Descifrado de Mensajes.....	12
3.2.5 RF-05 — Consulta de Directorio de Usuarios.....	12
3.2.6 RF-06 — Recuperación de Bandeja de Entrada	13
3.2.7 RF-07 — Cierre de Sesión Seguro	13
3.2.8 RF-08 — Migración Automática de Esquema de Base de Datos.....	13
3.2.9 RF-09 — Registro de Auditoría sin Contenido	14
3.3 Requisitos No Funcionales.....	14
3.3.1 Requisitos de Rendimiento	14
3.3.2 Seguridad.....	15
3.3.3 Fiabilidad.....	15
3.3.4 Disponibilidad	15
3.3.5 Mantenibilidad.....	16

3.3.6	<i>Portabilidad</i>	16
3.4	Otros Requisitos	16
3.4.1	<i>Requisitos de Conformidad Legal y Académica</i>	16
3.4.2	<i>Requisitos de Documentación</i>	16
4	Apéndices	17
	Apéndice A — Tabla de Análisis de Amenazas STRIDE	17
	Apéndice B — Protocolo de Mensajes JSON (TipoPaquete)	17
	Apéndice C — Artefactos Generados por el Sistema	18
	Apéndice D — Requisitos del Entorno de Compilación y Despliegue	18

1 Introducción

La presente Especificación de Requisitos de Software (SRS) documenta de forma completa y estructurada los requisitos funcionales y no funcionales del Sistema de Mensajería Cifrada Distribuida (SMCD), desarrollado en Java como proyecto en el curso Sistemas Operativos II de la Facultad de Ingeniería de la Universidad Panamericana. El documento sigue el estándar IEEE Std 830-1998 y sirve como guía técnica entre el equipo de desarrollo y los interesados del proyecto.

1.1 Propósito

Este documento tiene como propósito definir con precisión todos los requisitos que debe satisfacer el SMCD, proporcionando una base sólida para el diseño, implementación, pruebas y mantenimiento del sistema. Está dirigido a:

- Desarrolladores responsables de la implementación del sistema.
- Docentes y evaluadores académicos que validan el cumplimiento de requisitos.
- Administradores de sistemas que gestionan el despliegue en infraestructura de red.
- Usuarios finales interesados en comprender las capacidades y limitaciones del sistema.

1.2 Alcance

El Sistema de Mensajería Cifrada Distribuida es una aplicación cliente-servidor desarrollada en Java que permite el intercambio de mensajes privados con cifrado de extremo a extremo (E2E). El sistema garantiza la confidencialidad total de las comunicaciones, de modo que ni el servidor ni ningún intermediario pueda acceder al contenido de los mensajes. El SMCD comprende:

- Un servidor TLS centralizado que gestiona autenticación, sesiones y enrutado de mensajes cifrados.
- Un cliente de escritorio con interfaz gráfica Swing para usuarios finales en entornos con GUI.
- Un cliente de consola (modo headless) para entornos de contenedores o servidores sin GUI.
- Un módulo común compartido que define el protocolo de comunicación, las primitivas criptográficas y la configuración TLS.

El sistema no gestiona llamadas de voz, videollamadas ni transferencia de archivos binarios. Tampoco implementa una PKI pública; la validación de certificados se realiza mediante huella SHA-256.

1.3 Personal Involucrado

Campo	Descripción
Nombre	Eliseo Velásquez Rodas
Rol	Analistas, Diseñadores y Desarrolladores del SMCD
Categoría profesional	Estudiante de Ingeniería en Ciencias y Sistemas
Responsabilidades	Análisis de requisitos, diseño arquitectónico, implementación, pruebas y documentación del sistema
Información de contacto	Facultad de Ingeniería — Universidad Panamericana

Aprobación	Ingeniera Yolanda Estebán Chilel
-------------------	----------------------------------

1.4 Definiciones, Acrónimos y Abreviaturas

Término	Definición
SMCD	Sistema de Mensajería Cifrada Distribuida — el producto descrito en este documento
SRS	Software Requirements Specification — Especificación de Requisitos de Software
E2E	Cifrado de Extremo a Extremo: los mensajes se cifran en el cliente y solo se descifran en el cliente destinatario
TLS	Transport Layer Security — protocolo criptográfico para comunicaciones seguras sobre TCP
RSA-2048	Algoritmo de cifrado asimétrico con clave de 2048 bits para intercambio de claves de sesión
AES-256-GCM	Advanced Encryption Standard con clave de 256 bits en modo Galois/Counter Mode para cifrado simétrico autenticado
BCrypt	Función de hash adaptativa para almacenamiento seguro de contraseñas (factor de costo 12)
JWT/UUID	Token UUID generado por el servidor al autenticar una sesión; requerido en operaciones protegidas
JDBC	Java Database Connectivity — API estándar de Java para acceso a bases de datos relacionales
STRIDE	Modelo de amenazas: Spoofing, Tampering, Repudiation, Information Disclosure, DoS, Elevation of Privilege
OWASP ASVS	Application Security Verification Standard — marco de referencia para verificación de seguridad en aplicaciones
NIST	National Institute of Standards and Technology — organismo emisor de estándares criptográficos
RFC 8446	Request for Comments que define el protocolo TLS versión 1.3
PBKDF2	Password-Based Key Derivation Function 2 — función para derivar claves criptográficas a partir de contraseñas
SQLite	Motor de base de datos embebida, sin servidor, utilizado para persistencia local del SMCD

1.5 Referencias

Ref.	Título	Ruta / URL	Fecha	Autor
[1]	IEEE Std 830-1998 — Recommended Practice for SRS	https://standards.ieee.org	1998	IEEE
[2]	RFC 8446 — TLS Protocol Version 1.3	https://tools.ietf.org/html/rfc8446	2018	IETF
[3]	NIST SP 800-38D — GCM and GMAC	https://csrc.nist.gov/publications/detail/sp/800-38d/final	2007	NIST
[4]	OWASP ASVS 4.0 — Application Security Verification Standard	https://owasp.org/www-project-asvs/	2021	OWASP
[5]	Stallings, W. — Cryptography and Network Security (8. ^a ed.)	Pearson Education, 2022	2022	Stallings
[6]	Goetz et al. — Java Concurrency in Practice	Addison-Wesley, 2006	2006	Goetz

1.6 Resumen

El presente documento se organiza en cuatro secciones principales. La Sección 1 (Introducción) establece el propósito, alcance, personal involucrado, terminología y referencias del documento. La Sección 2 (Descripción General) describe el contexto del sistema, sus funcionalidades principales, los perfiles de usuario, las restricciones de diseño y la evolución previsible. La Sección 3 (Requisitos Específicos) detalla de forma exhaustiva los requisitos de interfaces, los requisitos funcionales y los requisitos no funcionales del sistema. La Sección 4 (Apéndices) incluye información complementaria relevante para la comprensión del sistema.

2 Descripción General

2.1 Perspectiva del Producto

El SMCD es un sistema independiente y autocontenido que no forma parte de un sistema mayor preexistente. Está concebido como una plataforma de mensajería segura con capacidad de despliegue en entornos de red local (LAN) o en redes corporativas aisladas. Su arquitectura sigue el patrón cliente-servidor centralizado con comunicación cifrada sobre TCP/TLS. El servidor actúa exclusivamente como enrutador de blobs cifrados, sin capacidad para acceder al contenido de los mensajes, garantizando privacidad incluso ante un compromiso del servidor.

El sistema se integra con la infraestructura de virtualización VMware Workstation para simular entornos de red distribuida, y opcionalmente con Docker Compose para despliegue en contenedores en entornos headless. No requiere conexión a Internet ni servicios externos de terceros.

2.2 Funcionalidad del Producto

Las funcionalidades principales del SMCD son las siguientes:

- Registro de usuarios: alta de nuevos usuarios con generación automática del par de claves RSA-2048, almacenamiento de la clave pública en el servidor y cifrado de la clave privada en el cliente con PBKDF2 + AES-256-GCM.
- Autenticación segura: verificación de credenciales mediante comparación BCrypt-12 y emisión de token UUID de sesión.
- Mensajería cifrada E2E: envío y recepción de mensajes con cifrado híbrido RSA-2048 + AES-256-GCM; el servidor solo maneja blobs cifrados con prefijo E2E1:.
- Push en tiempo real: entrega inmediata de mensajes a clientes conectados sin polling activo.
- Bandeja de entrada: recuperación del historial de mensajes cifrados del usuario autenticado.
- Gestión de usuarios: consulta del directorio de usuarios registrados con sus claves públicas.
- Cierre de sesión seguro: invalidación explícita del token UUID en el servidor.
- Monitor de administración: interfaz gráfica Swing en el servidor para supervisión de conexiones activas y auditoría.
- Registro de auditoría: log de eventos de conexión, autenticación y envío, sin incluir contenido de mensajes.

2.3 Características de los Usuarios

Atributo	Usuario Final	Administrador del Sistema
Formación	Sin conocimientos técnicos requeridos; uso mediante GUI Swing intuitiva	Conocimientos de administración de redes, Java y sistemas GNU/Linux
Habilidades	Manejo básico de computadora; capacidad de ejecutar scripts .bat	Gestión de VMs VMware, Docker, Gradle y configuración de red TCP/IP
Actividades	Registro, autenticación, envío y recepción de mensajes cifrados	Despliegue del servidor, gestión de certificados TLS, monitoreo y auditoría

2.4 Restricciones

- Lenguaje de implementación: Java 11 LTS (OpenJDK o Oracle JDK) obligatorio en servidor y clientes.
- Sistema de construcción: Gradle 8.7 mediante el wrapper gradlew; Gradle 9.x no es compatible por eliminación de APIs deprecadas.
- Motor de base de datos: SQLite embebido; no se admite un servidor de base de datos externo (MySQL, PostgreSQL, etc.).
- Interfaz gráfica: Swing (Java AWT/Swing); no se requiere instalación de JavaFX ni frameworks de terceros para la GUI.
- Certificados TLS: certificado autofirmado con validación por huella SHA-256; no se soporta validación mediante cadena de confianza de una CA pública sin modificación del código.

- Entorno de despliegue principal: VMware Workstation 17.x con VMs Ubuntu Desktop; el modo Docker Compose está limitado a clientes sin GUI (HEADLESS=true).
- Algoritmos criptográficos: los algoritmos RSA-2048, AES-256-GCM y BCrypt-12 no son negociables; no se admite downgrade a algoritmos de menor seguridad.

2.5 Suposiciones y Dependencias

- Se asume que la red donde se despliega el sistema es una red de confianza controlada (LAN corporativa o entorno virtualizado); el sistema no está diseñado para operación directa en Internet sin protección perimetral adicional.
- Se asume disponibilidad de Java JDK 11 o superior en todos los nodos (servidor y clientes).
- La disponibilidad de la biblioteca jBCrypt 0.4 y el driver SQLite JDBC 3.x es gestionada automáticamente por Gradle mediante los repositorios configurados.
- El correcto funcionamiento del modo GUI depende de la disponibilidad de un servidor X11 o entorno gráfico equivalente en la VM; cualquier cambio hacia entornos puramente headless requiere la activación del modo HEADLESS=true.
- Si el sistema operativo subyacente actualiza la JVM a una versión que deprecie APIs de javax.net.ssl o java.security utilizadas, podrían requerirse ajustes en el módulo common.

2.6 Evolución Previsible del Sistema

- Implementación del protocolo Double Ratchet para forward secrecy a nivel de aplicación, eliminando la dependencia de claves RSA estáticas por usuario.
- Soporte para grupos de mensajería con cifrado de clave de sesión por miembro del grupo.
- Migración de la función de hash de contraseñas de BCrypt a Argon2id, conforme a las recomendaciones actuales de OWASP.
- Implementación de una infraestructura PKI interna con autoridad certificadora propia, reemplazando la validación por huella SHA-256.
- Desarrollo de clientes para plataformas móviles (Android/iOS) manteniendo el mismo protocolo de comunicación JSON Lines sobre TLS.
- Integración con un sistema SIEM para correlación de eventos de auditoría y detección de anomalías.

3 Requisitos Específicos

Esta sección contiene la especificación completa y detallada de todos los requisitos del SMCD. Los requisitos funcionales (RF) y no funcionales (RNF) están identificados unívocamente para facilitar su trazabilidad, seguimiento y validación durante el desarrollo y las pruebas de aceptación.

3.1 Requisitos Comunes de los Interfaces

El SMCD define interfaces con el usuario, el hardware subyacente, componentes de software de terceros y los protocolos de comunicación de red. A continuación se especifican los requisitos de cada categoría de interfaz.

3.1.1 Interfaces de Usuario

La interfaz de usuario comprende dos modalidades operativas diferenciadas:

- GUI Swing para clientes: ventana principal con panel de selección de destinatario (lista de usuarios), área de visualización de mensajes recibidos (desencriptados en tiempo real), campo de entrada de texto y botón de envío. La interfaz debe cargarse completamente en menos de 3 segundos en hardware con JVM 11+ y 512 MB de RAM disponibles.
- Monitor Swing para el servidor: ventana de supervisión con tabla de conexiones activas (usuario, IP, timestamp de conexión), área de log en tiempo real con entradas coloreadas por nivel (INFO, WARN, ERROR) y controles de administración.
- Consola headless para cliente Docker: salida estándar con mensajes formateados que incluyen timestamp, remitente y contenido descifrado; entrada mediante stdin con prompts claros para cada operación.
- Scripts de lanzamiento: iniciar-servidor.bat / iniciar-servidor.sh, iniciar-cliente.bat / iniciar-cliente.sh, iniciar-distribuido.bat, ver-admin-base-datos.bat con mensajes de error descriptivos ante fallos de configuración.

3.1.2 Interfaces de Hardware

- El sistema requiere una tarjeta de red TCP/IP funcional en cada nodo (servidor y clientes) con soporte para sockets de red estándar de Java (java.net.Socket, javax.net.ssl.SSLSocket).
- Almacenamiento en disco: mínimo 50 MB libres para el servidor (base de datos, logs, artefactos); mínimo 20 MB para cada cliente (claves privadas cifradas, artefactos de compilación).
- Memoria RAM mínima: 256 MB disponibles para el servidor; 128 MB para cada instancia de cliente.
- No se requiere hardware criptográfico dedicado (HSM); toda la operación criptográfica se realiza por software mediante la JVM.

3.1.3 Interfaces de Software

Componente	Versión	Propósito e Interfaz
Java JDK	11 LTS o superior	Plataforma de ejecución; javax.net.ssl para TLS, java.security para RSA/AES, java.sql para JDBC
Gradle	8.7 (via wrapper)	Compilación multi-módulo, gestión de dependencias (jBCrypt, SQLite JDBC), generación de artefactos JAR
SQLite JDBC	3.x (latest)	Driver JDBC para acceso a la base de datos SQLite embebida; interfaz java.sql.Connection, PreparedStatement
jBCrypt	0.4	Biblioteca para hashing BCrypt; interfaz: BCrypt.hashpw(password, BCrypt.gensalt(12)), BCrypt.checkpw(candidate, hash)
VMware Workstation	17.x (opcional)	Virtualización de red para despliegue distribuido; red NAT o Host-Only para comunicación entre VMs
Docker + Compose	24.x (opcional)	Contenerización para modo headless; docker-compose.yml define servicios de servidor y cliente(s) consola

3.1.4 Interfaces de Comunicación

- Protocolo de transporte: TCP sobre TLS 1.2/1.3 (preferencia TLS 1.3 conforme RFC 8446) en el puerto 9443.
- Formato de mensajes: JSON Lines (un objeto JSON por línea, terminado en \n) con codificación UTF-8; campo discriminador tipo de tipo enumerado TipoPaquete.
- Validación de certificado: comparación de huella SHA-256 del certificado del servidor contra el archivo certs/active-cert.sha256 en cada cliente; conexión rechazada si no coincide.
- Tipos de paquete cliente → servidor: REGISTRO, INICIO_SESION, ENVIAR, BANDEJA, USUARIOS, CIERRE_SESION.
- Tipos de paquete servidor → cliente: OK, ERROR, MENSAJE, LISTA_USUARIOS.
- Serialización/deserialización: biblioteca estándar org.json o equivalente incluida en el módulo common; todos los campos de paquete son strings JSON.

3.2 Requisitos Funcionales

A continuación se especifican los requisitos funcionales del SMCD. Cada requisito está identificado con un código único (RF-XX), su nombre, tipo, fuente y prioridad.

3.2.1 RF-01 — Registro de Usuario con Generación de Claves RSA

Número de requisito	RF-01
Nombre de requisito	Registro de usuario con generación de claves RSA-2048
Tipo	Requisito funcional
Fuente del requisito	Análisis de seguridad E2E; NIST SP 800-175B; OWASP ASVS 4.0 §6
Prioridad del requisito	Alta / Esencial

El sistema debe permitir el registro de nuevos usuarios mediante los siguientes pasos:

- (1) el cliente genera un par de claves RSA-2048 mediante `java.security.KeyPairGenerator` con proveedor BC o `SunRsaSign`;
- (2) el cliente cifra la clave privada utilizando una clave derivada de la contraseña del usuario mediante PBKDF2-SHA256 con al menos 100 000 iteraciones y la cifra con AES-256-GCM con IV aleatorio de 96 bits, almacenando el resultado en `data/claves/{usuario}.key`;
- (3) el cliente envía al servidor un paquete REGISTRO con los campos: usuario (único, minúsculas), contraseña (texto plano para hash en servidor) y clavePublica (Base64 de la clave pública);
- (4) el servidor valida unicidad del nombre de usuario, genera el hash BCrypt con factor 12 y almacena (usuario, hash_contrasena, clave_publica, creado_en) en la tabla usuarios;
- (5) el servidor responde OK si el registro es exitoso o ERROR con mensaje descriptivo si el usuario ya existe.

3.2.2 RF-02 — Autenticación con Emisión de Token de Sesión

Número de requisito	RF-02
Nombre de requisito	Autenticación segura con emisión de token UUID de sesión
Tipo	Requisito funcional
Fuente del requisito	OWASP ASVS 4.0 §2; análisis de amenazas STRIDE — Spoofing
Prioridad del requisito	Alta / Esencial

El sistema debe autenticar usuarios registrados mediante los siguientes pasos:

- (1) el cliente envía un paquete INICIO_SESION con los campos usuario y contraseña;
- (2) el servidor recupera el hash BCrypt almacenado para el usuario indicado;
- (3) el servidor compara la contraseña recibida con el hash mediante BCrypt.checkpw();
- (4) si la comparación es exitosa, el servidor genera un token UUID v4 aleatorio, lo registra en GestorSesiones junto al nombre de usuario y responde OK con el campo token;
- (5) si la comparación falla o el usuario no existe, el servidor responde ERROR sin revelar cuál de las dos condiciones se cumple (prevención de enumeración de usuarios);
- (6) el cliente almacena el token en memoria para adjuntarlo en las operaciones subsiguientes.

3.2.3 RF-03 — Envío de Mensaje con Cifrado E2E

Número de requisito	RF-03
Nombre de requisito	Envío de mensaje privado con cifrado extremo a extremo
Tipo	Requisito funcional
Fuente del requisito	RFC 8446; NIST SP 800-38D; análisis de amenazas STRIDE — Information Disclosure
Prioridad del requisito	Alta / Esencial

El sistema debe permitir el envío de mensajes cifrados E2E mediante el siguiente proceso:

- (1) el cliente recupera la clave pública RSA del destinatario (obtenida previamente mediante RF-05 o en el registro);
- (2) el cliente genera una clave AES-256 aleatoria de sesión y un IV aleatorio de 96 bits;
- (3) el cliente cifra el texto del mensaje con AES-256-GCM obteniendo el ciphertext y el authentication tag;
- (4) el cliente cifra la clave AES de sesión con la clave pública RSA-2048 del destinatario usando padding OAEP-SHA256;
- (5) el cliente serializa el blob cifrado con el prefijo E2E1: seguido de la concatenación Base64 de (IV || AES_key_cifrada_RSA || ciphertext || auth_tag);
- (6) el cliente envía al servidor un paquete ENVIAR con los campos token, destinatario y contenido (el blob E2E1:);

(7) el servidor valida el token, almacena el blob cifrado en la tabla mensajes y, si el destinatario está conectado, hace push inmediato del paquete MENSAJE;

(8) si el destinatario no está conectado, el mensaje queda almacenado para entrega posterior mediante BANDEJA.

3.2.4 RF-04 — Recepción y Descifrado de Mensajes

Número de requisito	RF-04
Nombre de requisito	Recepción y descifrado local de mensajes E2E
Tipo	Requisito funcional
Fuente del requisito	Análisis de seguridad E2E; NIST SP 800-38D
Prioridad del requisito	Alta / Esencial

El sistema debe permitir la recepción y descifrado de mensajes mediante los siguientes pasos:

(1) al recibir un paquete MENSAJE con un blob E2E1:, el cliente extrae y decodifica en Base64 los componentes: IV (12 bytes), clave AES cifrada RSA, ciphertext y auth tag;

(2) el cliente descifra la clave AES de sesión utilizando su clave privada RSA-2048 con OAEP-SHA256;

(3) el cliente descifra el ciphertext con AES-256-GCM verificando el authentication tag; si la verificación falla, el mensaje se descarta y se registra el error;

(4) el mensaje descifrado se muestra en la GUI o consola del cliente con el nombre del remitente y el timestamp del servidor;

(5) la clave privada RSA se carga desde el archivo .key local derivando la clave de descifrado mediante PBKDF2 con la contraseña del usuario, que nunca se almacena en disco.

3.2.5 RF-05 — Consulta de Directorio de Usuarios

Número de requisito	RF-05
Nombre de requisito	Consulta del directorio de usuarios registrados con claves públicas
Tipo	Requisito funcional
Fuente del requisito	Diseño del protocolo de aplicación; requisito de usabilidad
Prioridad del requisito	Alta / Esencial

El sistema debe permitir al cliente autenticado obtener el listado completo de usuarios registrados con sus claves públicas. El cliente envía un paquete USUARIOS con su token; el servidor valida el token y responde con un paquete LISTA_USUARIOS que incluye un array JSON con objetos {usuario, clavePublica} para cada usuario activo en el sistema, excluyendo el propio usuario solicitante. El cliente actualiza su directorio local de claves públicas con la información recibida.

3.2.6 RF-06 — Recuperación de Bandeja de Entrada

Número de requisito	RF-06
Nombre de requisito	Recuperación del historial de mensajes cifrados (bandeja de entrada)
Tipo	Requisito funcional
Fuente del requisito	Análisis de requisitos de usuario; necesidad de continuidad ante desconexión
Prioridad del requisito	Alta / Esencial

El sistema debe permitir al cliente recuperar los mensajes que le fueron enviados durante periodos de desconexión. El cliente envía un paquete BANDEJA con su token; el servidor recupera de SQLite todos los mensajes en la tabla mensajes donde destinatario = usuario_autenticado, ordenados por enviado_en ASC, y los retorna en paquetes MENSAJE individuales. Los mensajes no se eliminan de la base de datos al ser entregados; se marca el campo leído = 1.

3.2.7 RF-07 — Cierre de Sesión Seguro

Número de requisito	RF-07
Nombre de requisito	Cierre de sesión con invalidación de token de servidor
Tipo	Requisito funcional
Fuente del requisito	OWASP ASVS 4.0 §3; análisis de amenazas STRIDE — Elevation of Privilege
Prioridad del requisito	Media / Deseado

El sistema debe invalidar el token de sesión tanto en el cliente como en el servidor al cerrar sesión. El cliente envía un paquete CIERRE_SESSION con su token; el servidor elimina el token del registro de GestorSesiones y responde OK. El cliente elimina el token de memoria. Si el cliente se desconecta abruptamente sin enviar CIERRE_SESSION, el servidor detecta la IOException en el hilo lector e invalida el token automáticamente para evitar tokens huérfanos.

3.2.8 RF-08 — Migración Automática de Esquema de Base de Datos

Número de requisito	RF-08
Nombre de requisito	Migración automática del esquema de base de datos entre versiones
Tipo	Requisito funcional
Fuente del requisito	Gestión de ciclo de vida del software; compatibilidad con instalaciones previas

Prioridad del requisito	Media / Deseado
--------------------------------	-----------------

Al iniciar el servidor, el módulo BaseDatosSqlite debe detectar automáticamente si existen tablas del esquema legacy en inglés (users, messages). En caso afirmativo, debe:

- (1) crear las nuevas tablas en español (usuarios, mensajes) si no existen;
- (2) copiar todos los registros de users → usuarios y de messages → mensajes con mapeo de columnas correspondiente;
- (3) eliminar las tablas legacy;
- (4) registrar en el log de auditoría el número de registros migrados. El proceso debe ser atómico (envuelto en una transacción SQLite) para evitar pérdida de datos ante fallos parciales.

3.2.9 RF-09 — Registro de Auditoría sin Contenido

Número de requisito	RF-09
Nombre de requisito	Registro de auditoría de eventos del sistema sin exposición de contenido
Tipo	Requisito funcional
Fuente del requisito	Análisis de amenazas STRIDE — Repudiation; política de privacidad E2E
Prioridad del requisito	Alta / Esencial

El servidor debe mantener un registro de auditoría persistente en logs/sistema.log con las siguientes características:

- (1) cada entrada incluye timestamp ISO 8601, nivel (INFO/WARN/ERROR), identificador del hilo y descripción del evento;
- (2) se registran eventos de: nueva conexión TLS (IP y puerto origen), autenticación exitosa (usuario), fallo de autenticación (sin revelar si es usuario o contraseña incorrecta), envío de mensaje (remitente, destinatario, tamaño del blob, sin contenido), cierre de sesión (usuario), error de protocolo y arranque/parada del servidor;
- (3) el log NO debe incluir en ningún caso el contenido de mensajes, contraseñas, tokens UUID ni claves criptográficas.

3.3 Requisitos No Funcionales

3.3.1 Requisitos de Rendimiento

ID	Requisito	Métrica	Valor objetivo
RNF-P1	Tiempo de autenticación BCrypt	Latencia por operación	< 500 ms (factor 12)
RNF-P2	Tiempo de cifrado E2E (mensaje de 1 KB)	Latencia por operación	< 50 ms en cliente

RNF-P3	Clientes simultáneos soportados	Concurrencia máxima	≥ 50 sin degradación
RNF-P4	Latencia de entrega push (red local)	Tiempo extremo a extremo	< 200 ms (LAN)
RNF-P5	Handshake TLS 1.3	Tiempo de establecimiento	< 300 ms (LAN)
RNF-P6	Arranque del servidor	Tiempo hasta puerto 9443 activo	< 5 s

3.3.2 Seguridad

- RNF-S1: Todo el tráfico entre cliente y servidor debe estar cifrado mediante TLS 1.2 o 1.3 (preferencia TLS 1.3). No se admiten conexiones sin cifrado ni downgrade a SSL 3.0 o TLS 1.0/1.1.
- RNF-S2: Las contraseñas de usuario deben almacenarse exclusivamente como hashes BCrypt con factor de costo mínimo 12 (OWASP ASVS 4.0 §2.4). Nunca deben almacenarse en texto claro, con hash MD5 o SHA-1.
- RNF-S3: El contenido de los mensajes debe almacenarse en la base de datos exclusivamente como blobs cifrados E2E1. El servidor no debe tener acceso, en ninguna circunstancia, a las claves privadas de los clientes.
- RNF-S4: Las claves privadas RSA deben estar cifradas en disco con AES-256-GCM derivado de PBKDF2-SHA256 con mínimo 100 000 iteraciones y salt aleatorio almacenado junto a la clave cifrada.
- RNF-S5: El certificado TLS del servidor debe validarse en el cliente mediante comparación de huella SHA-256 contra el archivo certs/active-cert.sha256. Cualquier discrepancia debe abortar la conexión.
- RNF-S6: El sistema debe ser conforme con el modelo de amenazas STRIDE, mitigando cada categoría según lo documentado en la tabla de análisis de amenazas del Apéndice A.

3.3.3 Fiabilidad

- RNF-F1: El servidor debe recuperarse automáticamente de desconexiones abruptas de clientes sin degradar el servicio a los demás usuarios conectados. La pérdida de una conexión no debe generar tokens UUID huérfanos ni condiciones de carrera en GestorSesiones.
- RNF-F2: El acceso concurrente a la base de datos SQLite debe estar serializado mediante el objeto de bloqueo dedicado (bloqueo en BaseDatosSqlite), garantizando ausencia de condiciones de carrera en escrituras concurrentes.
- RNF-F3: El sistema debe tolerar al menos 3 fallos de autenticación consecutivos del mismo cliente sin bloquear el servicio para otros usuarios (aislamiento de fallos por conexión).

3.3.4 Disponibilidad

- RNF-D1: El servidor debe estar disponible el 99% del tiempo durante las sesiones de uso planificadas. La única causa admitida de indisponibilidad planificada es el reinicio para actualización de certificados TLS o migración de esquema de base de datos.
- RNF-D2: El sistema debe soportar el reinicio del servidor sin pérdida de los mensajes almacenados en SQLite ni de las claves públicas registradas de los usuarios.

- RNF-D3: Los clientes deben detectar la pérdida de conectividad con el servidor en un máximo de 30 segundos y notificar al usuario de forma clara mediante un mensaje de estado en la GUI o consola.

3.3.5 Mantenibilidad

- RNF-M1: El código fuente debe organizarse en tres módulos Gradle independientes (common, server, client) con responsabilidades bien delimitadas y sin dependencias circulares entre módulos.
- RNF-M2: Toda la lógica criptográfica debe estar encapsulada en clases especializadas dentro del módulo common, de modo que cualquier cambio de algoritmo (por ejemplo, migración de BCrypt a Argon2) requiera modificación únicamente en ese módulo.
- RNF-M3: Los parámetros de configuración (puerto del servidor, factor de costo BCrypt, ruta del certificado TLS, ruta de la base de datos) deben centralizarse en archivos de configuración externos y no estar hardcoded en el código fuente.
- RNF-M4: El mantenimiento del sistema (actualización de certificados, migración de esquema, rotación de logs) debe poder realizarse por el administrador sin modificar el código fuente, únicamente mediante scripts de administración provistos.

3.3.6 Portabilidad

- RNF-PO1: El sistema debe funcionar en cualquier sistema operativo que soporte Java JDK 11 LTS, incluyendo Windows 10/11, Ubuntu 20.04+, Debian 11+ y macOS 12+.
- RNF-PO2: El código fuente no debe contener dependencias de APIs específicas de un sistema operativo; las únicas excepciones admitidas son los scripts de lanzamiento (.bat para Windows, .sh para Unix), que deben mantenerse sincronizados en funcionalidad.
- RNF-PO3: El modo headless (HEADLESS=true) debe permitir la ejecución del cliente en cualquier entorno Linux sin servidor X11, incluyendo contenedores Docker basados en imágenes OpenJDK mínimas.
- RNF-PO4: La base de datos SQLite es portable entre plataformas; el archivo mensajeria.db debe poder copiarse entre sistemas operativos sin pérdida de integridad de los datos.

3.4 Otros Requisitos

3.4.1 Requisitos de Conformidad Legal y Académica

- RNF-L1: El sistema debe ser utilizado exclusivamente en entornos académicos y de investigación dentro de la Facultad de Ingeniería de UPANA; su uso en producción con datos de usuarios reales requiere una auditoría de seguridad independiente y conformidad con la legislación de protección de datos aplicable.
- RNF-L2: Todas las bibliotecas de terceros utilizadas (jBCrypt, SQLite JDBC) deben tener licencias compatibles con su uso académico y estar correctamente declaradas en el archivo build.gradle del módulo correspondiente.

3.4.2 Requisitos de Documentación

- RNF-DO1: El sistema debe incluir documentación técnica completa: manual de instalación y despliegue (VMware y Docker), manual de usuario (GUI y consola), informe técnico conforme IEEE 830 y documentación del protocolo de comunicación con descripción de todos los tipos de paquete.
- RNF-DO2: El código fuente debe incluir comentarios Javadoc en todas las clases y métodos públicos de los tres módulos, con descripción de propósito, parámetros y valores de retorno.

4 Apéndices

Apéndice A — Tabla de Análisis de Amenazas STRIDE

Amenaza STRIDE	Descripción	Impacto	Mitigación implementada
Spoofing	Suplantación de identidad de usuario	Acceso no autorizado al sistema	BCrypt-12 + Token UUID por sesión (RF-01, RF-02)
Tampering	Modificación de mensajes en tránsito	Mensajes alterados sin detección	TLS 1.3 + GCM Auth Tag 128 bits (RNF-S1, RF-03)
Repudiation	Negación del envío de un mensaje	Imposibilidad de atribuir acciones	Registro de auditoría con timestamps (RF-09)
Info. Disclosure	Acceso al contenido de mensajes por el servidor o BD	Exposición de comunicaciones privadas	Cifrado E2E RSA-2048 + AES-256-GCM (RF-03, RNF-S3)
Denial of Service	Saturación del servidor con conexiones masivas	Indisponibilidad del servicio	ExecutorService con pool dinámico; aislamiento por hilo (RNF-F1)
Elevation of Privilege	Acceso a operaciones sin autenticación válida	Ejecución de operaciones no autorizadas	Validación de token UUID en cada operación protegida (RF-02)

Apéndice B — Protocolo de Mensajes JSON (TipoPaquete)

TipoPaquete	Dir.	Campos principales	Autenticación requerida
REGISTRO	C→S	tipo, usuario, contraseña, clavePublica	No
INICIO_SESION	C→S	tipo, usuario, contraseña	No
ENVIAR	C→S	tipo, token, destinatario, contenido (E2E1:...)	Token UUID de sesión
BANDEJA	C→S	tipo, token	Token UUID de sesión
USUARIOS	C→S	tipo, token	Token UUID de sesión
CIERRE_SESION	C→S	tipo, token	Token UUID de sesión
OK	S→C	tipo, mensaje, [token] (en INICIO_SESION)	—
ERROR	S→C	tipo, mensaje (descripción del error)	—

MENSAJE	S→C	tipo, remitente, contenido (E2E1:...), timestamp	—
LISTA_USUARIOS	S→C	tipo, usuarios [{usuario, clavePublica}]	—

Apéndice C — Artefactos Generados por el Sistema

Artefacto	Módulo origen	Descripción
logs/sistema.log	server	Registro de auditoría sin contenido de mensajes
data/mensajeria.db	server	Base de datos SQLite con usuarios y mensajes cifrados
server.p12	server	Certificado TLS autofirmado del servidor (PKCS#12)
certs/active-cert.sha256	server	Huella SHA-256 del certificado para validación en clientes
data/claves/{usuario}.key	client	Clave privada RSA cifrada con PBKDF2+AES-256-GCM
server/build/libs/server.jar	server	JAR ejecutable del servidor TLS
client/build/libs/client.jar	client	JAR ejecutable del cliente (GUI + headless)

Apéndice D — Requisitos del Entorno de Compilación y Despliegue

Componente	Versión mínima	Tipo	Observación
Java JDK	11 LTS	Obligatorio	OpenJDK o Oracle JDK; requerido en todos los nodos
Gradle	8.7 (via gradlew)	Obligatorio	Gradle 9.x incompatible por APIs deprecadas eliminadas
SQLite JDBC	3.x latest	Auto (Gradle)	Gestionado por Gradle; no requiere instalación manual
jBCrypt	0.4	Auto (Gradle)	Biblioteca BCrypt para Java; gestionada por Gradle
VMware Workstation	17.x	Opcional	Para despliegue distribuido en VMs con GUI

Docker + Compose	24.x	Opcional	Para modo headless sin GUI; activar HEADLESS=true
------------------	------	----------	--

— Fin del documento ERS IEEE 830 — Sistema de Mensajería Cifrada Distribuida — Rev. 1.0 — Junio 2025 —